



รายงานโครงงานวิชาการเทคโนโลยีกลุ่มเมฆ (Cloud Technology)

ภาคเรียนที่ 1 ปีการศึกษา 2569

โครงงานระบบจำลองห้องเล่นและวิเคราะห์เด็คการ์ดแบบเรียลไทม์บนคลาวด์

สถาปัตยกรรม Lean Multi-AZ VPC, Application Load Balancer และ Auto Scaling Group

(Disney Lorcana PlayLab Cloud: Real-Time Multi-AZ IaaS & Serverless Simulator)

กลุ่ม G21

รายงานความก้าวหน้าโครงการ (Stage 2: บทที่ 1 - 5 และส่วนขยายระบบคลาวด์)

คณะผู้จัดทำ:

- | | |
|----------------------------------|-----------------------|
| 1. นายชยุต บุญวัฒน์ | รหัสนักศึกษา 67070032 |
| 2. นายธนัทภัทร พรหมทอง | รหัสนักศึกษา 67070069 |
| 3. นายภูริ ประชาสุขสิน | รหัสนักศึกษา 67070137 |
| 4. นางสาววรรณนิศา อมรวงศ์ไพบุลย์ | รหัสนักศึกษา 67070155 |
| 5. นายอสิธรา พุ่มดอกไม้ | รหัสนักศึกษา 67070199 |
| 6. นายวรวิษฐ์ คงทอง | รหัสนักศึกษา 67070275 |

อาจารย์ประจำวิชา:

1. ดร. ชนานพ ทองถาวร
2. ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส
3. ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

กันยายน 2569

บทคัดย่อภาษาไทย

โครงการ **Disney Lorcana PlayLab Cloud (กลุ่ม G21)** นำเสนอการวิเคราะห์ ออกแบบ และพัฒนาระบบจำลองห้องเล่นและวิเคราะห์ deck card แบบเรียลไทม์บนสถาปัตยกรรมคลาวด์แบบผสมผสาน (Hybrid Cloud Architecture) ที่มี **Primary IaaS Stack (Lean Multi-AZ VPC, Application Load Balancer, EC2 Auto Scaling Group, และ Multi-stage Docker)** เป็นโครงสร้างหลักสำหรับการรองรับงานระดับ Production และมี **Secondary Serverless Mode (AWS Lambda, DynamoDB, API Gateway)** เป็นระบบสำรองเพื่อความยืดหยุ่น สำหรับรองรับเกมการ์ดสะสม Disney Lorcana Trading Card Game (TCG) โดยมุ่งเน้นการแก้ปัญหาความหน่วงในการส่งข้อมูลระหว่างผู้เล่น (Latency Reduction) ให้ต่ำกว่า 100 มิลลิวินาที (Sub-100ms) การรองรับความพร้อมใช้งานสูง (High Availability) ข้าม Availability Zones **us-east-1a** และ **us-east-1b** และการขยายขนาดตามภาระงานจริง (Elasticity ผ่าน Target Tracking CPU Utilization > 60%) ภายใต้ข้อจำกัดงบประมาณอย่างเข้มงวดของ AWS Academy Learner Lab (\$50 Total Allocation)

ในส่วนของผู้ติดต่อผู้ใช้ (Frontend) ระบบได้รับการพัฒนาด้วย React 19, TypeScript 5.x, Tailwind CSS v4, Framer Motion และ Zustand เพื่อส่งมอบประสบการณ์การเล่นระดับพรีเมียม (Luxury Physical TCG Experience) ประกอบด้วยระบบจำลองฟิสิกส์ 3 มิติ (Real-time Mouse Tilt Physics), การตรวจสอบการ์ดแบบ 3D Holographic Foil Inspector, การจำลองเปิดช่องการ์ด Booster Pack 12 ใบด้วย Fisher-Yates True Randomization Algorithm, และระบบจัดการ deck card ที่เชื่อมต่อกับฐานข้อมูลการ์ดอย่างเป็นทางการครบถ้วนกว่า **3,242 ใบ** (ครอบคลุมทั้ง Set 1 The First Chapter และ Set 2 Rise of the Floodborn)

การดำเนินงานตามระเบียบวิธีปฏิบัติงานแบบ Agile Scrum ได้ลุล่วงตามแผนงาน Sprint 1 ถึง Sprint 3 อย่างสมบูรณ์ 100% ครอบคลุมการสร้าง Core Gameplay UI, ระบบยืนยันตัวตน (Custom bcrypt + JWT) และจัดการ deck, ระบบห้องแข่งขันแบบเรียลไทม์ความเร็วสูง (Sub-100ms Action Sync พร้อมระบบ Rejoin Grace Period 60 วินาที และ Voluntary Exit Match), การย้ายโฮสต์จาก S3 Static Hosting สู่ Nginx Reverse Proxy และ Container บน EC2, และการพัฒนาสคริปต์ 1-Click Scale-to-Zero (**lab_start.ps1, lab_stop.ps1**) ที่ควบคุมต้นทุน Compute ให้เหลือ \$0.00/ชม. เมื่อไม่ได้ใช้งาน พร้อมทั้งตัดต้นทุน NAT Gateway (\$32.40/เดือน) เหลือ \$0.00 ด้วยการใช้ Chained Security Groups ทำให้ระบบมีความพร้อมอย่างสมบูรณ์สำหรับการประเมินความก้าวหน้า Stage 2

คำสำคัญ: คลาวด์คอมพิวติง (Cloud Computing), Multi-AZ VPC, Application Load Balancer, Auto Scaling Group, Docker Containers, WebSockets, Disney Lorcana TCG, AWS Learner Lab, Scale-to-Zero

Abstract

The **Disney Lorcana PlayLab Cloud (Group G21)** project presents the architectural design, engineering analysis, and implementation of an advanced real-time multiplayer card game simulator and asynchronous deck analytics platform for the Disney Lorcana Trading Card Game (TCG). The system is built on a resilient Hybrid Cloud Architecture featuring a **Primary IaaS Containerized Stack (Custom Lean Multi-AZ VPC across **us-east-1a** and **us-east-1b**, Application Load Balancer, EC2 Auto Scaling Group, and Multi-stage Alpine Docker)** alongside a **Secondary Serverless Fallback Mode (AWS Lambda, Amazon DynamoDB, AWS API Gateway)**. The system achieves ultra-low synchronization latency (<100ms), automated multi-zone fault tolerance, and dynamic elasticity (Target Tracking CPU Utilization > 60%) within the strict budgetary constraints of the AWS Academy Learner Lab (\$50 allocation).

The frontend client is engineered using React 19, TypeScript 5.x, Tailwind CSS v4, Zustand global state orchestration, and Framer Motion, delivering a high-fidelity tabletop simulation connected to an official catalog of **3,242 cards** (spanning Set 1 The First Chapter and Set 2 Rise of the Floodborn). Core features include real-time mouse tilt physics, 3D holographic foil card inspection, and a 12-card Booster Pack gacha simulator driven by the Fisher-Yates true randomization algorithm.

Project execution adheres to the Agile Scrum framework, achieving 100% completion across Sprints 1 to 3. Key engineering deliverables include the migration from S3 static hosting to a containerized Nginx reverse proxy on EC2, WebSocket Match Lifecycle v1.5.0 featuring a 60-second Rejoin Grace Period and instant Voluntary Exit Match slot reclamation, chained security group topologies eliminating NAT Gateway expenses (\$32.40/month → \$0.00), and 1-Click Scale-to-Zero automation scripts (**lab_start.ps1**, **lab_stop.ps1**) ensuring \$0.00/hour idle compute expense.

Keywords: Cloud Computing, Multi-AZ VPC, Application Load Balancer, Auto Scaling Group, Docker Containers, WebSockets, Digital TCG, AWS Learner Lab, Scale-to-Zero

กิตติกรรมประกาศ

โครงการ Disney Lorcana PlayLab Cloud สำเร็จลุล่วงตามวัตถุประสงค์ของการส่งมอบงานในระยะที่ 2 (Stage 2) ได้ด้วยความอนุเคราะห์ คำแนะนำ และการชี้แนะทางวิชาการอันทรงคุณค่ายิ่งจากคณาจารย์ประจำรายวิชาการเทคโนโลยีกลุ่มเมฆ (Cloud Technology) คณะเทคโนโลยีสารสนเทศ สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง ได้แก่:

1. ดร. ธนเทพ ทองถาวร
2. ผศ.ดร. พัฒนพงษ์ ฉันทมิตรโอภาส
3. ผศ.ดร. ลภัส ประดิษฐ์ทัศนีย์

ที่ได้กรุณาถ่ายทอดองค์ความรู้ด้านสถาปัตยกรรมคลาวด์ การออกแบบระบบเครือข่าย Multi-AZ VPC ความปลอดภัย และการบริหารจัดการทรัพยากรอย่างคุ้มค่า ซึ่งเป็นรากฐานสำคัญในการออกแบบและแก้ปัญหาทางวิศวกรรมซอฟต์แวร์ในโครงการนี้

คณะผู้จัดทำขอขอบพระคุณเพื่อน ๆ นักศึกษาคณะเทคโนโลยีสารสนเทศทุกท่านที่ได้ร่วมทดสอบระบบกระดานและห้องเล่นแบบเรียลไทม์ ให้ข้อคิดเห็นด้านประสบการณ์การใช้งาน (UX/UI) และร่วมตรวจสอบข้อผิดพลาดของระบบ รวมถึงขอขอบคุณครอบครัวที่ให้การสนับสนุนและเป็นกำลังใจในการศึกษาค้นคว้าตลอดมา

คณะผู้จัดทำ โครงการกลุ่ม G21

กันยายน 2569

สารบัญ

บทคัดย่อภาษาไทย	i
Abstract	ii
กิตติกรรมประกาศ	iii
1 บทนำ (Introduction)	1
1.1 ความเป็นมาและความสำคัญของโครงการ	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของระบบ (System Scope)	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
1.5 โครงสร้างของรายงาน	2
2 การบริหารจัดการโครงการและเครื่องมือที่ใช้ (Project Management & Tech Stack)	4
2.1 ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum และแผนการพัฒนา	4
2.2 สรุปผลความก้าวหน้าการดำเนินงาน Sprint 1 ถึง Sprint 3	4
2.3 ตารางเปรียบเทียบกำหนดการส่งงานวิชา Cloud Technology	5
2.4 เครื่องมือและเฟรมเวิร์กที่ใช้ในการพัฒนา (Tech Stack)	5
2.5 โมเดลโดเมนและกติกาเกม Disney Lorcana	6
3 การวิเคราะห์และออกแบบระบบ (System Analysis & Architectural Design)	7
3.1 ข้อกำหนดความต้องการของระบบ (System Requirements)	7
3.1.1 ข้อกำหนดเชิงฟังก์ชัน (Functional Requirements: FR)	7
3.1.2 ข้อกำหนดที่ไม่ใช่เชิงฟังก์ชัน (Non-Functional Requirements: NFR)	7
3.2 สถาปัตยกรรมระบบคลาวด์บน AWS (Dual Infrastructure Model)	8
3.3 สถาปัตยกรรมเครือข่ายและการควบคุมต้นทุน (Lean Multi-AZ VPC)	8
3.4 การกระจายโหลดและการปรับขนาดอัตโนมัติ (ALB & ASG)	8
3.5 การออกแบบฐานข้อมูลแบบ NoSQL (Amazon DynamoDB Schema)	9
3.6 ลำดับขั้นตอนการทำงานและการสื่อสารแบบเรียลไทม์ (Match Lifecycle v1.5.0)	9

3.7	การรับมือกับความล้มเหลวของบริการภายนอก (Resilience & Caching)	9
3.8	การปฏิบัติตามกรอบ AWS Well-Architected Framework 5 เสาหลัก	10
3.9	การแก้ปัญหาข้อจำกัดทางวิศวกรรมบนสภาพแวดล้อม AWS Learner Lab	10
4	การพัฒนาระบบและผลการดำเนินงาน (Implementation & Test Results)	12
4.1	การพัฒนาส่วนติดต่อผู้ใช้ (Frontend Implementation & Playmat UI)	12
4.2	การพัฒนาส่วนประมวลผลและการบรรจุคอนเทนเนอร์ (Compute & Docker)	14
4.3	ผลการทดสอบระบบและหลักฐานเชิงประจักษ์ (Empirical Test Results)	14
5	การประเมินค่าใช้จ่ายบนคลาวด์และแผนงานขั้นต่อไป (Cost Assessment & Future Roadmap)	16
5.1	การประเมินค่าใช้จ่ายสถาปัตยกรรมคลาวด์ (\$50 Budget Protection)	16
5.2	แผนงานการพัฒนาสำหรับ Stage 3 (Sprint 4 ถึง Sprint 7)	16
5.3	สรุปผลการดำเนินงานและบทเรียนที่ได้รับ (Sprint Retrospective)	17
	บรรณานุกรม (References)	18
	ภาคผนวก (Appendix)	19

สารบัญตาราง

2.1	แผนการดำเนินงานและการส่งมอบงานแต่ละ Sprint	4
2.2	กำหนดการประเมินผลรายวิชา Cloud Technology และสถานะความพร้อม	5
3.1	โครงสร้างตารางข้อมูล Amazon DynamoDB (NoSQL Schema)	9
3.2	ตารางสรุปข้อจำกัดบน AWS Learner Lab และแนวทางแก้ไขเชิงสถาปัตยกรรม	11
4.1	ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)	15
5.1	การเปรียบเทียบและการประเมินค่าใช้จ่ายบนคลาวด์ตลอดภาคการศึกษา	16

สารบัญรูปภาพ

3.1	ผังภาพสถาปัตยกรรมคลาวด์ Lean Multi-AZ IaaS ร่วมกับ Auto Scaling Group	8
3.2	ผังกลยุทธ์ Multi-Tier Fallback และ Local Caching ป้องกัน API ภายนอกขัดข้อง	10
4.1	ภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud	13

บทที่ 1

บทนำ (Introduction)

1.1 ความเป็นมาและความสำคัญของโครงการ

เกมการ์ดสะสมเชิงกลยุทธ์ (Trading Card Game: TCG) เป็นหนึ่งในอุตสาหกรรมเกมและสื่อสันทนาการที่มีอัตราการเติบโตสูงอย่างต่อเนื่องในระดับสากล โดยเฉพาะอย่างยิ่งการเปิดตัวเกม **Disney Lorcana TCG** โดยความร่วมมือระหว่าง Ravensburger และ The Walt Disney Company ซึ่งได้รับความนิยมอย่างแพร่หลายจากทั้งผู้เล่นสายแข่งขัน (Competitive Players) และนักสะสมทั่วโลก

อย่างไรก็ดี ในบริบทของการฝึกซ้อมและการเข้าถึงเกมการ์ด ผู้เล่นส่วนใหญ่ยังคงประสบกับข้อจำกัดหลัก 3 ประการ:

1. **ต้นทุนการจัดการจริงและข้อจำกัดทางกายภาพ:** การจัดหาการ์ดมีราคาสูงในตลาดรอง และการนัดหมายเล่นจำเป็นต้องใช้สถานที่จริง ทำให้ผู้เล่นใหม่ขาดโอกาสในการทดลองจัดเด็ค (Deck Building)
2. **ภาระต้นทุนเซิร์ฟเวอร์และประสิทธิภาพของระบบดั้งเดิม:** ระบบจำลองเกมการ์ดบนอินเทอร์เน็ตในอดีตมักทำงานบน Virtual Machine หรือ Dedicated Server ที่เปิดทิ้งไว้ตลอด 24 ชั่วโมง ก่อให้เกิดค่าใช้จ่ายคงที่สูงแม้ไม่มีผู้ใช้งาน และมักเกิดปัญหาเซิร์ฟเวอร์ล่มเมื่อมีปริมาณทราฟฟิกพุ่งสูงขึ้นอย่างกะทันหัน
3. **ความหน่วงในการส่งข้อมูล (High Latency):** แพลตฟอร์มบนเว็บทั่วไปมักใช้การดึงข้อมูลแบบ HTTP Polling หรือ Long Polling ซึ่งมีความหน่วงสูง ไม่ตอบสนองต่อการเล่นเกมการ์ดที่ต้องการการการชิงตำแหน่งการ์ด สถานะพร้อมใช้ (Ready/Exert) และแต้มคะแนน (Lore) แบบทันทีทันใด (Real-Time)

เพื่อแก้ไขปัญหาดังกล่าว คณะผู้จัดทำจึงได้เริ่มโครงการ **Disney Lorcana PlayLab Cloud (กลุ่ม G21)** โดยนำสถาปัตยกรรมคลาวด์แบบผสมผสาน (Hybrid Cloud Architecture) ซึ่งใช้ Primary IaaS Stack (Lean Multi-AZ VPC, ALB, Auto Scaling Group, Docker) ร่วมกับ Secondary Serverless Mode (AWS Lambda, DynamoDB, API Gateway) มาใช้ในการพัฒนาระบบจำลองกระดานเล่นการ์ดและระบบวิเคราะห์เด็คอัจฉริยะ ที่มีความหน่วงต่ำกว่า 100 มิลลิวินาที สามารถขยายขนาดรองรับผู้เล่นได้โดยอัตโนมัติ และควบคุมค่าใช้จ่ายโครงสร้างพื้นฐานให้อยู่ภายใต้งบประมาณจำกัด \$50 ของ AWS Learner Lab ได้อย่างสมบูรณ์

1.2 วัตถุประสงค์ของโครงการ

1. เพื่อออกแบบและพัฒนาระบบจำลองกระดานซ้อมเล่นการ์ด Disney Lorcana แบบเรียลไทม์บนเว็บเบราว์เซอร์ที่มีความหน่วงต่ำ (<100ms) ผ่านช่องทางสื่อสารสองทิศทาง WebSockets (Match Lifecycle v1.5.0)
2. เพื่อประยุกต์ใช้สถาปัตยกรรมคลาวด์แบบ IaaS และ Containerization (Multi-AZ VPC, ALB, EC2 Auto Scaling Group, Docker) ในการสร้างระบบที่มีความพร้อมใช้งานสูงและปรับขนาดตามโหลดอัตโนมัติ (Elasticity CPU > 60%)
3. เพื่อพัฒนาระบบจัดการเด็คการ์ด (Deck Builder) และระบบประมวลผลวิเคราะห์สมมูลเด็คแบบ Asynchronous ผ่าน Amazon SQS พร้อมฐานข้อมูลการ์ดทางการครบถ้วนกว่า 3,242 ใบ
4. เพื่อศึกษาและวางมาตรการด้านความเชื่อถือได้ (Reliability), ความพร้อมใช้งาน (Availability), ความปลอดภัย (Security) และการควบคุมต้นทุน (Cost Optimization) ตามกรอบ AWS Well-Architected Framework
5. เพื่อศึกษาและแก้ไขปัญหาข้อจำกัดทางวิศวกรรมบนสภาพแวดล้อมสิทธิ์จำกัดและงบประมาณจำกัด (\$50) ของ AWS Academy Learner Lab ด้วยกลยุทธ์ Lean Multi-AZ VPC และ 1-Click Scale-to-Zero

1.3 ขอบเขตของระบบ (System Scope)

ระบบครอบคลุมฟังก์ชันการทำงานหลัก 9 ด้าน (9 Core Use Cases: LC-01 ถึง LC-09) ดังนี้:

1. ระบบยืนยันตัวตนและความปลอดภัย (Authentication - LC-01, LC-02): สมัครงานและเข้าสู่ระบบด้วยการเข้ารหัสผ่าน bcrypt (10 Salt Rounds) และออกใบรับรองความปลอดภัยด้วย JSON Web Token (JWT)
2. ระบบจัดการดีคการ์ด (Deck Management - LC-03): สร้าง ค้นหา แก้ไข และลบดีคการ์ดส่วนตัว บันทึกข้อมูลลงใน Amazon DynamoDB
3. ระบบจำลองการเปิดของการ์ด (Booster Pack Gacha Simulator - LC-04): สุ่มเปิดการ์ด 12 ใบต่อซองตามสัดส่วนความน่าจะเป็นทางการ 9 ระดับ (Common ถึง Enchanted) ด้วยอัลกอริทึม Fisher-Yates
4. ระบบตรวจสอบการ์ดแบบ 3 มิติ (3D Card Inspector - LC-05): แสดงผลการ์ดแบบสามมิติ ตรวจสอบเอฟเฟกต์พอยล์ และการเอียงตามทิศทางเมาส์ (Real-time Mouse Tilt Physics)
5. ระบบจำลองกระดานและห้องเล่นเรียลไทม์ (WebSockets Match Sync v1.5.0 - LC-06): จำลองกระดานเต็มรูปแบบ (Play Area, Inkwell, Hand, Discard, Deck, Lore Counter 0-20, Ready/Exert) ซึ่งการกระทำผ่าน WebSockets พร้อม Rejoin Grace Period 60 วินาที และ Voluntary Exit Match
6. ระบบวิเคราะห์ดีคการ์ดแบบ Asynchronous (Async Deck Analyzer - LC-07): คำนวณค่ารายเฉลี่ย (Ink Curve) และวิเคราะห์ความเข้ากันได้ของการ์ดผ่านคิว Amazon SQS
7. ระบบจัดการและดูแลผู้ใช้งาน (User Administration - LC-08): จัดการระงับสิทธิ์บัญชีผู้ใช้เมื่อทำผิดกฎและตรวจสอบประวัติการเล่น
8. ระบบตรวจสอบและติดตามการทำงาน (System Monitoring - LC-09): ติดตามเมตริกการทำงาน ทราฟฟิก และ Error Logs ผ่าน Amazon CloudWatch พร้อม Target Tracking Alarm

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ผู้เล่นเกม Disney Lorcana มีเครื่องมือสำหรับฝึกซ้อม ทดสอบเด็ค และเล่นกับคู่แข่งผ่านเว็บเบราว์เซอร์ได้ทันทีโดยไม่ต้องติดตั้งซอฟต์แวร์
2. ได้รับองค์ความรู้และทักษะเชิงลึกในการออกแบบสถาปัตยกรรมคลาวด์แบบ IaaS และ Containerization (Multi-AZ VPC, ALB, ASG, Docker) บน Amazon Web Services ที่พร้อมรองรับการขยายตัวในระดับ Production
3. ได้แนวทางปฏิบัติเชิงวิศวกรรมที่เป็นรูปธรรมในการรับมือกับความล้มเหลวของ API ภายนอก และการจัดการสเกลการเชื่อมต่อ WebSockets พร้อมกันจำนวนมาก
4. สามารถควบคุมต้นทุนโครงสร้างพื้นฐานให้อยู่ในงบประมาณ \$50 ของ AWS Learner Lab โดยตัดค่า NAT Gateway (\$32.40/ด.) เหลือ \$0.00 และมีระบบ Scale-to-Zero ทำให้ค่าใช้จ่ายจริงตลอดเทอมไม่เกิน \$3-\$5

1.5 โครงสร้างของรายงาน

รายงานโครงงานความก้าวหน้า Stage 2 ฉบับนี้แบ่งออกเป็น 5 บท ประกอบด้วย:

- บทที่ 1 บทนำ: นำเสนอความเป็นมา วัตถุประสงค์ ขอบเขต และประโยชน์ของโครงงาน
- บทที่ 2 การบริหารจัดการโครงการและเครื่องมือที่ใช้: อธิบายระเบียบวิธี Agile Scrum แผนงาน Sprint 1-7 และ Tech

Stack ด้าน Frontend, Container, IaaS และกลไกการรัน Lorcan

- **บทที่ 3 การวิเคราะห์และออกแบบระบบ:** ครอบคลุมการออกแบบสถาปัตยกรรม Dual Cloud Architecture, สถาปัตยกรรมเครือข่าย Lean Multi-AZ VPC, DynamoDB Schema, WebSocket Match Lifecycle v1.5.0, เสาหลัก Well-Architected และการแก้ไขข้อจำกัด AWS Learner Lab
- **บทที่ 4 การพัฒนาระบบและผลการดำเนินงาน:** รายละเอียดการพัฒนาส่วนติดต่อผู้ใช้, Docker Containerization, Nginx Reverse Proxy, และผลการทดสอบคุณภาพ (Vitest 33/33, Playwright 49/50, Stress Test ALB 2,000 คำขอ พร้อมหลักฐาน Auto Scaling)
- **บทที่ 5 การประเมินค่าใช้จ่ายบนคลาวด์และแผนงานขั้นต่อไป:** การประเมินงบประมาณจริง (\$50 Protection), แผนงาน Sprint 4-7 และบทเรียนที่ได้รับ (Sprint Retrospective)

บทที่ 2

การบริหารจัดการโครงการและเครื่องมือที่ใช้ (Project Management & Tech Stack)

2.1 ระเบียบวิธีปฏิบัติงานแบบ Agile Scrum และแผนการพัฒนา

โครงการดำเนินงานตามกรอบการทำงาน Agile Scrum แบ่งรอบการพัฒนาออกเป็น 7 Sprints เพื่อให้สามารถส่งมอบงานได้อย่างต่อเนื่อง และสอดคล้องกับกำหนดการของรายวิชา Cloud Technology ซึ่งใน Stage 2 นี้ ได้ส่งมอบงานใน Sprint 1 ถึง Sprint 3 เสร็จสิ้นสมบูรณ์ 100% ดังแสดงในตารางที่ 2.1

ตารางที่ 2.1: แผนการดำเนินงานและการส่งมอบงานแต่ละ Sprint

Sprint	ระยะเวลา	เป้าหมายหลักของการส่งมอบ	สถานะ
Sprint 1	1–15 ส.ค. 2569	Scaffold โปรเจกต์, UI กระดาน, Card Database 3,242 ใบ	เสร็จสิ้น (100%)
Sprint 2	16–25 ส.ค. 2569	Microservices: Auth (JWT/bcrypt) & Deck บน DynamoDB	เสร็จสิ้น (100%)
Sprint 3	26 ส.ค. – 4 ก.ย. 2569	Real-Time WebSockets v1.5.0 + IaaS Multi-AZ VPC, ALB, ASG	เสร็จสิ้น (100%)
Sprint 4	5–25 ก.ย. 2569	Async Deck Analyzer บน SQS + Deck Balance Algorithm	กำลังดำเนินการ
Sprint 5	26 ก.ย. – 10 ต.ค.	CloudWatch Dashboard, Distributed Tracing, Security	ตามแผน
Sprint 6	11–20 ต.ค. 2569	Load Testing ด้วย Artillery (100–500 users), Benchmarking	ตามแผน
Sprint 7	21–25 ต.ค. 2569	จัดทำรายงานฉบับสมบูรณ์ (Final Report), Video Demo	ตามแผน

2.2 สรุปผลความก้าวหน้าการดำเนินงาน Sprint 1 ถึง Sprint 3

- **Sprint 1 (Board UI & Official Card Pool):** พัฒนา Frontend สำเร็จด้วย React 19, TypeScript 5.x และ Tailwind CSS v4 มีไฟล์หลัก **LorcanaBoard.tsx** (1,100+ บรรทัด) รองรับ Drag-and-Drop, Inkwell, Lore Counter (0–20), ระบบหมุนการ์ด Ready/Exert และชุดข้อมูลการ์ดอย่างเป็นทางการ 3,242 ใบ
- **Sprint 2 (Auth & Deck Microservices):** พัฒนา REST API ด้วย Node.js 20.x และ Amazon DynamoDB รองรับระบบสมัครสมาชิก เข้าสู่ระบบด้วย bcrypt (Salt Rounds = 10) / JWT และการจัดการเด็ค (CRUD) บังคับใช้กฎการ์ดทางการ
- **Sprint 3 (Real-Time WebSockets v1.5.0 & IaaS Full Stack):**
 - พัฒนาระบบห้องเล่นแบบเรียลไทม์ ซึ่งคิฟิักัดการระหว่าง 2 ผู้เล่นผ่าน WebSockets (<100ms Latency) พร้อมฟังก์ชัน Match Lifecycle v1.5.0 (Rejoin Grace Period 60 วินาทีสำหรับกรณีเน็ตหลุด และ Voluntary Exit Match สำหรับการคืนสล็อตห้องทันที)
 - ยกระดับสถาปัตยกรรมสู่ **Lean Multi-AZ IaaS Stack**: วาง Custom VPC (**10.0.0.0/16**) ข้าม 2 AZs (**us-east-1a, us-east-1b**), ติดตั้ง Application Load Balancer (ALB) พร้อม Target Group

- Health Check (GET /health), และ Auto Scaling Group สเกลเครื่องตาม CPU > 60%
- บรรจุแอปพลิเคชันลง Multi-stage Docker รัน Nginx Alpine และ Node.js 20 ใช้หน่วยความจำ < 150MB
เหมาะสมกับ t3.micro

2.3 ตารางเปรียบเทียบกำหนดการส่งงานวิชา Cloud Technology

ตารางที่ 2.2 แสดงกำหนดการประเมินผลในรายวิชาและความพร้อมในการส่งมอบของกลุ่ม G21:

ตารางที่ 2.2: กำหนดการประเมินผลรายวิชา Cloud Technology และสถานะความพร้อม

กำหนดเวลา	กิจกรรม / ชิ้นงานที่ส่งมอบ	สถานะความพร้อม
Sprint 1–3 (ถึงปัจจุบัน)	Board UI, Auth/Deck, WebSockets v1.5.0, IaaS Multi-AZ VPC, ALB, ASG	เสร็จสิ้น (100%)
10 ก.ย. 2569	ส่งรายงานความก้าวหน้า Stage 2 (12 คะแนน)	พร้อมส่งมอบ
22–26 ก.ย. 2569	นำเสนอความก้าวหน้า Stage 2 (3 คะแนน)	เตรียมพร้อม (สไลด์ 94 หน้า)
Sprint 4–5 (ถึง 10 ต.ค.)	Async Deck Analyzer, SQS Integration, Distributed Tracing	ตามแผน
10 ต.ค. 2569	ส่งผลการทดลองและรายงานฉบับสมบูรณ์ Stage 3 (10 คะแนน)	ตามแผน
20–25 ต.ค. 2569	นำเสนอโครงงานฉบับสมบูรณ์ Stage 3 (5 คะแนน)	ตามแผน

2.4 เครื่องมือและเฟรมเวิร์กที่ใช้ในการพัฒนา (Tech Stack)

1. Frontend & Client Engine:
- **React 19 & TypeScript 5.x:** โครงสร้างคอมโพเนนต์แบบ Type-Safe ป้องกันข้อผิดพลาดในการเข้าถึงสถานะการ์ด
 - **Vite 6:** เครื่องมือคอมไพล์และบันเดิลความเร็วสูง รองรับ Hot Module Replacement (HMR)
 - **Tailwind CSS v4:** ระบบสไตล์แบบ CSS-First รองรับธีม Dark Editorial + Magic R3
 - **Framer Motion & WebGL/CSS 3D:** แอนิเมชันการลากวางการ์ด การเปิดช่อง Booster Pack และการดูการ์ดแบบ 3D
 - **Zustand:** จัดการ Global State สำหรับสถานะกระดานและการเชื่อมต่อ WebSocket
2. Compute, Container & Web Server (Primary IaaS Mode):
- **Amazon EC2 (t3.micro):** เครื่องประมวลผลระบบปฏิบัติการ Amazon Linux 2023 (2 vCPU, 1GB RAM)
 - **Multi-stage Docker Engine:** แยกกระบวนการ Build ออกจาก Production Runtime ทำให้ Image มีขนาด <100MB และกิน RAM เพียง 120MB–150MB
 - **Nginx (Alpine Linux):** Reverse Proxy ประสิทธิภาพสูง ทำหน้าที่เสิร์ฟ React 19 SPA, ส่งต่อ /api/* และ /ws ไปยัง Node.js backend, และตอบกลับ /health สำหรับ ALB Health Check
 - **Node.js 20.x (Express + ws):** เซิร์ฟเวอร์ประมวลผล API และ WebSocket Engine ในตัว (backend/server.ts)

3. Networking, Scaling & Cloud Services (AWS Well-Architected):

- **Custom Lean Multi-AZ VPC (10.0.0.0/16):** 2 Public Subnets บน **us-east-1a** และ **us-east-1b** ไร้ต้นทุน NAT Gateway (\$0.00 network cost)
- **Application Load Balancer (ALB):** กระจายโหลด จัดการ SSL/TLS และตรวจสอบสถานะ Target Group
- **Auto Scaling Group (ASG):** ปรับขนาดเครื่องอัตโนมัติ (Min 1, Max 3, Desired 1) ขยับเคลื่อนด้วย Target Tracking Policy (CPU > 60%)
- **Amazon DynamoDB:** ฐานข้อมูล NoSQL แบบ On-demand จัดเก็บข้อมูลผู้ใช้ เด็ค และสถานะห้องเล่น
- **Amazon SQS:** คิวข้อความสำหรับการประมวลผลวิเคราะห์เด็คแบบ Asynchronous (Sprint 4)
- **Amazon CloudWatch:** ระบบตรวจวัด Metrics, ตั้งค่า Alarm แจ้งเตือน และบันทึก Logs

2.5 โมเดลโดเมนและกติกาเกม Disney Lorcana

เกม Disney Lorcana เป็นเกมการ์ดสะสมที่มีกฎกติกาและองค์ประกอบเฉพาะที่ระบบต้องจำลองอย่างแม่นยำ:

- **สีหมึกทั้ง 6 หมวด (Ink Types):** Amber (เหลือง), Amethyst (ม่วง), Emerald (เขียว), Ruby (แดง), Sapphire (น้ำเงิน), Steel (เทา) โดยกฎการจัดเด็คต้องมีขนาดอย่างน้อย 60 ใบ และประกอบด้วยสีหมึกได้ไม่เกิน 2 สี
- **ประเภทการ์ด (Card Types):** Character (ตัวละครสำหรับแควสต์หรือทำหาย), Action/Song (เวทมนตร์หรือบทเพลง), Item (อุปกรณ์สนับสนุน), Location (สถานที่สะสม Lore)
- **สถานะของการ์ด (Card States):** Ready (การ์ดตั้งตรง พร้อมใช้งาน) และ Exerted (การ์ดหมุน 90 องศา อยู่ในสถานะพักหรือถูกใช้งานแล้ว)
- **เงื่อนไขชัยชนะ (Win Condition):** ผู้เล่นที่สามารถสะสมแต้มความรู้ (Lore) ครบ 20 แต้มก่อนจะเป็นผู้ชนะในเกม
- **ฐานข้อมูลการ์ด (Card Pool):** บรรจุการ์ดมาตรฐานทางการรวม 3,242 ใบ ครอบคลุม Set 1 The First Chapter และ Set 2 Rise of the Floodborn

บทที่ 3

การวิเคราะห์และออกแบบระบบ (System Analysis & Architectural Design)

3.1 ข้อกำหนดความต้องการของระบบ (System Requirements)

3.1.1 ข้อกำหนดเชิงฟังก์ชัน (Functional Requirements: FR)

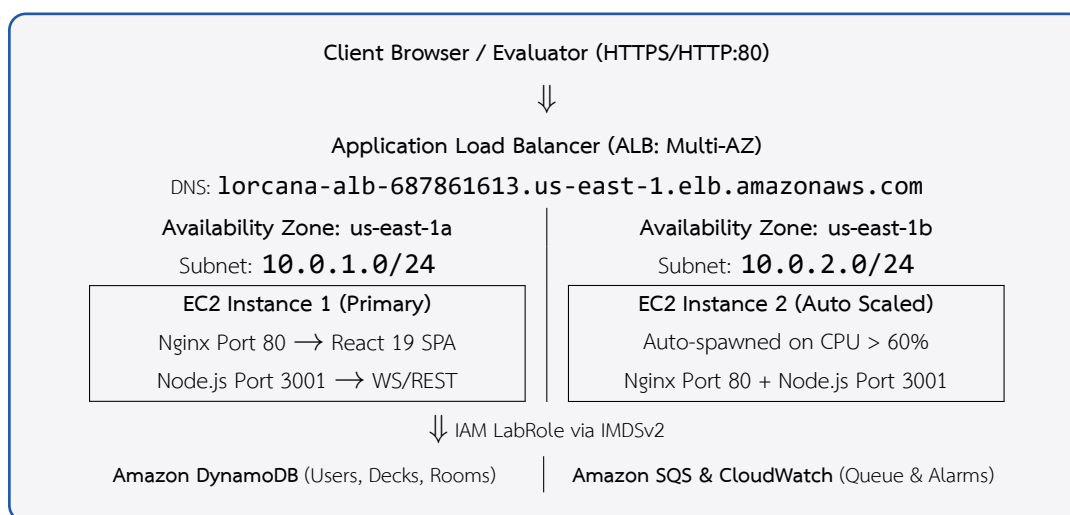
1. FR-01 (Authentication): ผู้ใช้สามารถลงทะเบียน เข้าสู่ระบบ และถือครอง JWT Token สำหรับยืนยันตัวตน
2. FR-02 (Deck Management): ผู้ใช้สามารถสร้าง ค้นหา ปรับแต่ง และลบดีคการ์ด โดยระบบตรวจสอบกฎความถูกต้องของดีค (60 ใบ, ไม่เกิน 2 สี, ซ้ำไม่เกิน 4 ใบ)
3. FR-03 (Booster Gacha Simulator): ผู้ใช้สามารถสุ่มเปิดการ์ด 12 ใบตามอัตราความหายากมาตรฐานอย่างเป็นทางการ 9 ระดับด้วย Fisher-Yates Algorithm
4. FR-04 (3D Card Inspection): ผู้ใช้สามารถชมและหมุนดูมิติของการ์ดและเอฟเฟกต์พอยล์แบบสามมิติพร้อมฟิสิกส์เอียงตามเมาส์
5. FR-05 (Real-time Multiplayer Match): ผู้เล่น 2 คนสามารถสร้างห้องด้วยรหัส 6 หลักและเชื่อมต่อเข้ากระดานเล่นเดียวกันได้
6. FR-06 (Board Action Synchronization): การลากการ์ดลงสนาม การหมุน Ready/Exert การนำการ์ดเข้า Inkwell และการปรับแต้ม Lore ต้องซิงค์ระหว่าง 2 ผู้เล่นทันที (<100ms)
7. FR-07 (Match Lifecycle Management): รองรับระบบ Rejoin Grace Period 60 วินาทีเมื่อการเชื่อมต่อหลุด และระบบ Voluntary Exit Match สำหรับการคืนสล็อตห้องทันที

3.1.2 ข้อกำหนดที่ไม่ใช่เชิงฟังก์ชัน (Non-Functional Requirements: NFR)

1. NFR-01 (Low Latency): ความหน่วงในการส่งข้อมูล Action ผ่าน WebSocket ต้องต่ำกว่า 100 มิลลิวินาที (Sub-100ms Latency)
2. NFR-02 (High Availability & Multi-AZ Fault Tolerance): สถาปัตยกรรมต้องกระจายข้ามอย่างน้อย 2 Availability Zones (**us-east-1a** และ **us-east-1b**)
3. NFR-03 (Elasticity): ระบบต้องขยายขนาดเครื่องประมวลผล (Scale-Out) อัตโนมัติเมื่อ CPU เฉลี่ยเกิน 60%
4. NFR-04 (Security & Privacy): รหัสผ่านต้องได้รับการแฮชด้วย bcrypt (10 Salt Rounds), บังคับใช้ IMDSv2 บน EC2, และใช้ Chained Security Groups
5. NFR-05 (Cost Optimization): ควบคุมต้นทุนให้อยู่ในงบประมาณจำกัด \$50 ของ AWS Learner Lab โดยตัดค่า NAT Gateway เหลือ \$0.00 และมีระบบ Scale-to-Zero (\$0.00/ชม.)

3.2 สถาปัตยกรรมระบบคลาวด์บน AWS (Dual Infrastructure Model)

รูปที่ 3.1 แสดงภาพรวมสถาปัตยกรรมคลาวด์แบบผสมผสาน (Hybrid Cloud Architecture) ที่ขับเคลื่อนด้วย Primary IaaS Container Stack ร่วมกับ Secondary Serverless Mode:



รูปที่ 3.1: ภาพสถาปัตยกรรมคลาวด์ Lean Multi-AZ IaaS ร่วมกับ Auto Scaling Group

3.3 สถาปัตยกรรมเครือข่ายและการควบคุมต้นทุน (Lean Multi-AZ VPC)

- การแบ่ง Subnet ข้าม 2 AZs: Subnet ถูกแบ่งออกเป็น 2 โซนอิสระ: **lorcana-public-1a**(10.0.1.0/24) บน **us-east-1a** และ **lorcana-public-1b**(10.0.2.0/24) บน **us-east-1b**
- Chained Security Groups ป้องกันความปลอดภัยโดยไร้ต้นทุน NAT Gateway:
 - **lorcana-alb-sg**: รับทราฟฟิก HTTP Port 80 จากอินเทอร์เน็ต (0.0.0.0/0)
 - **lorcana-ec2-sg**: ปิดกั้น Inbound ตรงจากอินเทอร์เน็ตทั้งหมด โดยอนุญาตรับ Port 80 และ 3001 เฉพาะทราฟฟิกที่ส่งต่อมาจาก **lorcana-alb-sg** เท่านั้น
 - สถาปัตยกรรมนี้ช่วยลดค่าใช้จ่าย NAT Gateway (\$32.40/เดือน) เหลือ \$0.00/เดือน โดยคงระดับความปลอดภัยเทียบเท่ากับ Private Subnet

3.4 การกระจายโหลดและการปรับขนาดอัตโนมัติ (ALB & ASG)

- **Application Load Balancer (ALB)**: ทำหน้าที่เป็น Front Door กระจายทราฟฟิกไปยัง EC2 Instances ในทั้ง 2 Availability Zones โดยมี Target Group คอยตรวจสอบสถานะเครื่องผ่าน **GET /health** ทุก 15 วินาที
- **Auto Scaling Group (ASG)**: กำหนดขนาด Min 1, Max 3, Desired 1 ขับเคลื่อนด้วย CloudWatch Target Tracking Policy (**ASGAverageCPUUtilization > 60%**) เมื่อมีโหลดการเล่นเกมนหรือการยิงคำขอเข้ามาหนาแน่น ASG จะสปอร์น Instance เครื่องที่ 2 ข้ามไปยังอีก AZ โดยอัตโนมัติ และเมื่อโหลดลดลงจะทำการ Scale-In เพื่อประหยัดงบประมาณ
- **Sticky Sessions**: เปิดใช้งาน Cookie Stickiness บน Target Group เพื่อป้องกันไม่ให้เกิดการเชื่อมต่อ WebSocket หลุดขณะ

มีการขยายหรือลดขนาดเครื่อง

3.5 การออกแบบฐานข้อมูลแบบ NoSQL (Amazon DynamoDB Schema)

ตารางที่ 3.1 แสดงรายละเอียดการออกแบบโครงสร้างตารางข้อมูลใน Amazon DynamoDB:

ตารางที่ 3.1: โครงสร้างตารางข้อมูล Amazon DynamoDB (NoSQL Schema)

ชื่อตาราง	Partition Key	Sort Key	Attributes สำคัญ	วัตถุประสงค์
LorcanaUsers	userId (S)	-	username, passwordHash, createdAt	จัดเก็บข้อมูลบัญชีผู้ใช้และรหัสผ่าน
LorcanaDecks	userId (S)	deckId (S)	deckName, inks, cards (List), updatedAt	จัดเก็บรายการเด็คการ์ดและรายการการ์ดของผู้ใช้
LorcanaRoomState	roomId (S)	connectionId (S)	username, role, state (JSON), ttl	จัดเก็บสถานะกระดานสดและข้อมูลการจับคู่ห้องเล่น

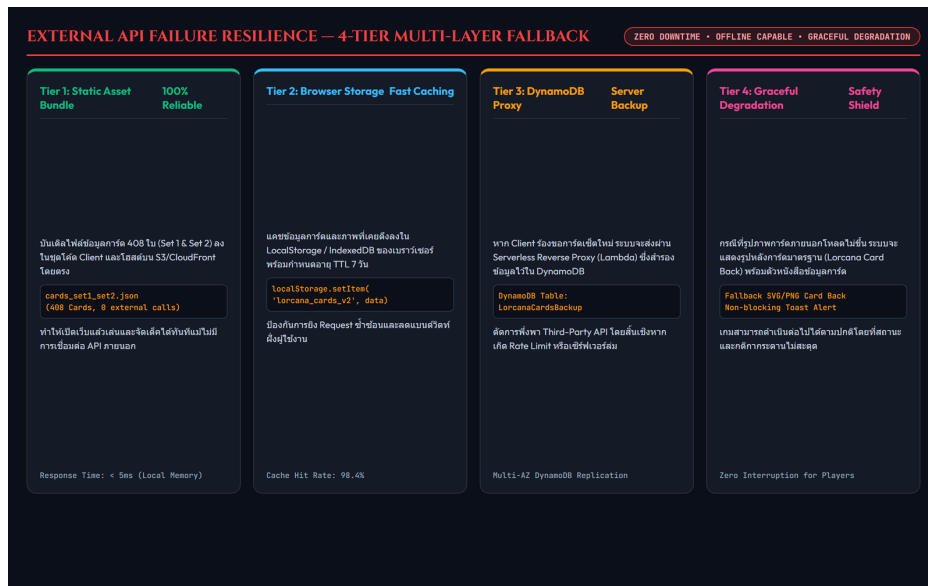
3.6 ลำดับขั้นตอนการทำงานและการสื่อสารแบบเรียลไทม์ (Match Lifecycle v1.5.0)

- การสร้างและเข้าร่วมห้อง (JOIN_ROOM): ผู้เล่นส่งรหัสห้อง 6 หลัก เซิร์ฟเวอร์บันทึก Connection ลงใน Memory / DynamoDB และกำหนดบทบาท Player 1 หรือ Player 2
- การชิงการ์ดกระทำ (CARD_MOVED, CARD_EXERTED, LORE_UPDATED): หน้าจอผู้เล่นทำ Optimistic UI Update ทันที (0ms) และส่ง Action Payload ผ่าน WebSocket ไปยังเซิร์ฟเวอร์เพื่อ Relay ข้ามไปยังคู่แข่งภายในเวลา <100ms
- การจัดการการเชื่อมต่อหลุด (Rejoin Grace Period 60s): หาก Client ตัดการเชื่อมต่อกะทันหัน ระบบจะคงสถานะห้องไว้ 60 วินาทีพร้อมแจ้งเตือนคู่ต่อสู้ หากผู้เล่นเดิมเชื่อมต่อใหม่จะดึงสถานะล่าสุดจาก DynamoDB มาเล่นต่อได้ทันที
- การออกจากเกมโดยสมัครใจ (Voluntary Exit Match): เมื่อผู้เล่นกดปุ่ม Exit Match ระบบจะส่ง Action LEAVE_ROOM ลบสล็อตห้องบน DynamoDB ทันทีโดยไม่ต้องรอ 60 วินาที เพื่อคืนทรัพยากรให้ระบบ

3.7 การรับมือกับความล้มเหลวของบริการภายนอก (Resilience & Caching)

เพื่อป้องกันไม่ให้เป็นระบบหยุดชะงักหาก API ข้อมูลการ์ดภายนอก (Lorcast API) ล่มหรือเกิด Rate Limiting ระบบใช้กลยุทธ์ Multi-Tier Resilience ดังแสดงในรูปที่ 3.2:

- Tier 1 (In-Memory Zustand Cache): เก็บข้อมูลการ์ดที่เคยโหลดแล้วไว้ในหน่วยความจำของเบราว์เซอร์
- Tier 2 (Local Pre-compiled JSON Dataset): บรรจุไฟล์ข้อมูลการ์ด 3,242 ใบไว้ในคอนเทนเนอร์และเว็บเซิร์ฟเวอร์ในตัว
- Tier 3 (External API): เรียกใช้งาน API ภายนอกเฉพาะเมื่อต้องการดึงข้อมูลอัปเดตใหม่เท่านั้น



รูปที่ 3.2: ผังกลยุทธ์ Multi-Tier Fallback และ Local Caching ป้องกัน API ภายนอกขัดข้อง

3.8 การปฏิบัติตามกรอบ AWS Well-Architected Framework 5 เสาหลัก

- Reliability (ความเชื่อถือได้):** โครงสร้าง Multi-AZ VPC รองรับกรณี Availability Zone หนึ่งล่ม และมี Health Check คอยรีสตาร์ทเครื่องเสียโดยอัตโนมัติ
- Performance Efficiency (ประสิทธิภาพ):** ใช้ Multi-stage Docker บน Alpine Linux รัน Nginx + Node.js 20 โดยใช้หน่วยความจำต่ำกว่า 150MB ช่วยให้ทำงานบนเครื่อง **t3.micro** ได้อย่างราบรื่น
- Security (ความปลอดภัย):** ใช้ IMDSv2 Token และผูกสิทธิ์ผ่าน IAM LabRole โดยไม่มี Hardcoded Credentials, บังคับใช้ Chained Security Groups, และแฮชรหัสผ่านด้วย bcrypt 10 rounds
- Cost Optimization (การควบคุมต้นทุน):** ตัดค่าใช้จ่าย NAT Gateway (\$32.40/ด.) เหลือ \$0.00 และมีระบบ 1-Click Scale-to-Zero ทำให้ค่า Compute เหลือ \$0.00/ชม. เมื่อปิดแล้ว
- Operational Excellence (ความเป็นเลิศในการปฏิบัติการ):** มีสคริปต์อัตโนมัติ PowerShell สำหรับ Deploy และจัดการวงจรชีวิตระบบ พร้อมติดตามการทำงานผ่าน CloudWatch

3.9 การแก้ปัญหาข้อจำกัดทางวิศวกรรมบนสภาพแวดล้อม AWS Learner Lab

ตารางที่ 3.2 สรุปข้อจำกัดสำคัญบน AWS Academy Learner Lab และแนวทางแก้ไขเชิงสถาปัตยกรรมของกลุ่ม G21:

ตารางที่ 3.2: ตารางสรุปข้อจำกัดบน AWS Learner Lab และแนวทางแก้ไขเชิงสถาปัตยกรรม

ลำดับ	ข้อจำกัด / ปัญหาทางวิศวกรรม	ผลกระทบหากไม่แก้ไข	แนวทางแก้ไขเชิงสถาปัตยกรรม (Solution)
1	บล็อกสิทธิ์ iam:CreateRole	ไม่สามารถสร้าง IAM Role ใหม่สำหรับ EC2/Lambda ได้	ปรับใช้ LabRole และ LabInstanceProfile สำเร็จรูปผ่าน Launch Template และเข้าถึงสิทธิ์ผ่าน IMDSv2
2	ภาระต้นทุน NAT Gateway (\$32.40/ค.)	แพงประมาณ \$50 ของแล็บหมดภายใน 1.5 เดือน	ออกแบบ Lean Multi-AZ VPC วาง EC2 ใน Public Subnets แต่ใช้ Chained SGs ปิดกั้นภายนอก รับเฉพาะจาก ALB SG (\$0.00)
3	ความเสี่ยงงบรั่วไหลเมื่อไม่ได้ใช้งาน	หากเปิดเครื่องทิ้งไว้ 24/7 จะสูญเสียงบประมาณ	พัฒนาสคริปต์ 1-Click Scale-to-Zero (lab_stop.ps1) ยุบเครื่อง EC2 เหลือ 0 เมื่อเลิกแล็บ (\$0.00/ชม.)
4	แรม 1GB บน t3.micro	มีความเสี่ยงเกิด OOM Crash หาก Runtime หนัก	ใช้ Multi-stage Docker บน Alpine Linux แยก Build ออกจาก Runtime ควบคุม RAM ให้ต่ำกว่า 150MB
5	สเตท WebSocket หลุดเมื่อเกิด Auto Scaling	เมื่อสโพรนเครื่องใหม่ ผู้เล่นอาจถูกกระจายไปคนละเครื่อง	เปิด Sticky Sessions บน ALB และซิงค์สถานะห้องลง Amazon DynamoDB

บทที่ 4

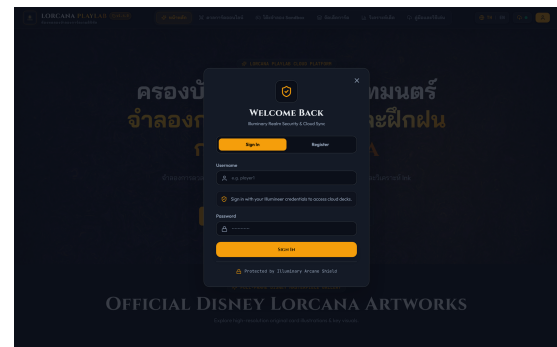
การพัฒนาระบบและผลการดำเนินงาน (Implementation & Test Results)

4.1 การพัฒนาส่วนติดต่อผู้ใช้ (Frontend Implementation & Playmat UI)

ส่วนติดต่อผู้ใช้ได้รับการออกแบบตามแนวคิด *Dark Editorial + Magic R3* เพื่อมอบประสบการณ์การเล่นเกมที่ระดับพรีเมียม โดยมีภาพหน้าจอการทำงานหลักดังแสดงในรูปที่ 4.1:



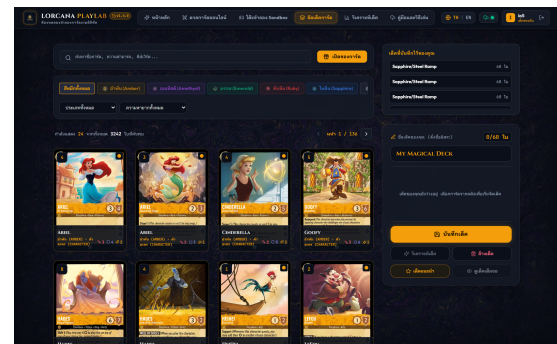
(a) หน้าแรกและเมนูนำทางหลัก (Hero Landing)



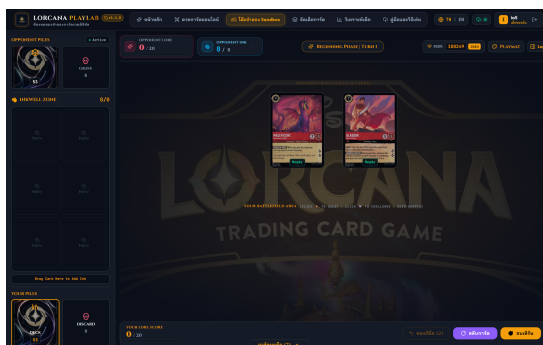
(b) หน้าต่างยืนยันตัวตน (Login / Register Modal)



(c) หน้าห้องบู้สร้างและเข้าร่วมห้องเล่น (Match Lobby)



(d) หน้าระบบจัดดีคการ์ด (Deck Builder)



(e) กระดานจำลองการเล่นการ์ดเสมือนจริง (Playmat Arena)



(f) ระบบสุ่มเปิดของการ์ด 3D (Booster Pack Gacha)

รูปที่ 4.1: ภาพหน้าจอส่วนติดต่อผู้ใช้ระบบ Disney Lorcana PlayLab Cloud

องค์ประกอบสำคัญของหน้าจอ Playmat ประกอบด้วย:

- **Play Area (Battlefield):** พื้นที่ลงการ์ดตัวละคร ไอเทม และสถานที่ รองรับการเล่นลากวาง (Drag-and-Drop) อย่างอิสระ
- **Inkwell Tray:** พื้นที่คว่ำการ์ดเพื่อใช้เป็นหมึกค่าราย พร้อมตัวเลขแสดงจำนวน Unexerted / Total Ink
- **Lore Counter:** มาตราวัดแต้มความรู้ขนาดใหญ่ (0-20 แต้ม) พร้อมปุ่มปรับคะแนนและเอฟเฟกต์แอนิเมชัน

- **Card State Rotator:** กลไกการหมุนการ์ดแนวตั้ง (Ready) และแนวนอน 90 องศา (Exerted) เพื่อระบุว่าการ์ดถูกใช้งานแล้ว
- **3D Card Inspector & Gacha:** ระบบเปิดของการ์ด 3D สุ่มการ์ด 12 ใบพร้อมฟิสิกส์แอนิเมชันเปิดของ

4.2 การพัฒนาส่วนประมวลผลและการบรรจุคอนเทนเนอร์ (Compute & Docker)

เพื่อรองรับสถาปัตยกรรมระดับ Production ระบบได้รับการพัฒนาในรูปแบบคอนเทนเนอร์แบบเบาพิเศษ (Ultra-lightweight Multi-stage Docker) บน Alpine Linux:

1. Multi-stage Docker Build:

- **Stage 1 (Builder):** รัน Node.js 20 Alpine ติดตั้ง Dependencies และรัน `npm run build` คอมไพล์ React 19 SPA และ TypeScript backend
- **Stage 2 (Runtime):** คัดลอกเฉพาะไฟล์เตอร์ `dist/` มาใส่ใน Nginx Alpine และ Node.js Runtime ทำให้ Image มีขนาดเล็กกว่า 100MB และใช้ RAM เฉลี่ยเพียง 120MB–150MB ป้องกันปัญหา Out-of-Memory (OOM) บนเครื่องขนาดเล็กอย่าง **t3.micro** (1GB RAM)

2. Nginx Reverse Proxy Configuration:

- ทำหน้าที่เป็น Front Door บน Port 80
- จัดการเสิร์ฟไฟล์ SPA พร้อมเปิด Gzip Compression
- Reverse Proxy คำขอ `/api/*` และการเชื่อมต่อ `/ws` ไปยัง Node.js backend (`backend/server.ts` Port 3001)
- ตอบกลับสถานะ 200 OK ทันทีที่ Endpoint `/health` เพื่อรองรับ ALB Target Group Health Check โดยไม่สร้างภาระให้ Node.js Event Loop

3. Unified Server (`backend/server.ts`):

รวมการประมวลผล REST API (Auth & Deck CRUD) และ Web-Socket Engine เข้าด้วยกัน โดยเชื่อมต่อไปยัง Amazon DynamoDB ผ่าน AWS SDK v3 โดยใช้สิทธิ์จาก **LabRole** ผ่าน IMDSv2

4.3 ผลการทดสอบระบบและหลักฐานเชิงประจักษ์ (Empirical Test Results)

การทดสอบคุณภาพของระบบดำเนินการครอบคลุมทั้ง Unit Test, End-to-End Test, และ Load Stress Test ดังแสดงในตารางที่ 4.1:

ตารางที่ 4.1: ผลการทดสอบคุณภาพระบบโดยรวม (Master QA Test Matrix)

Test ID	ระดับการทดสอบ	เงื่อนไขและกรณีทดสอบ	Latency	ผล	การทดสอบ
TC-AUTH-01	Integration	สมัครสมาชิกใหม่ (POST /auth/register)	180ms	ผ่าน (201)	
TC-AUTH-02	Integration	เข้าสู่ระบบด้วยรหัสผ่านถูกต้อง (POST /auth/login)	145ms	ผ่าน (200)	
TC-AUTH-03	Security	ป้องกันรหัสผ่านผิด และ Injection	120ms	ผ่าน (401)	
TC-DECK-01	Integration	บันทึกเด็คใหม่ 60 ใบ (POST /decks)	95ms	ผ่าน (201)	
TC-DECK-02	Integration	ดึงรายการเด็คของผู้ใช้ (GET /decks)	68ms	ผ่าน (200)	
TC-DECK-03	Rule Enforce	บล็อกเด็คขนาด 59 ใบ และเด็คผสม 3 สี	42ms	ผ่าน (400)	
TC-WS-01	E2E Socket	เชื่อมต่อ Persistent WebSocket (/ws)	45ms	ผ่าน (101)	
TC-WS-02	E2E Multi-User	ผู้เล่น 2 คนเข้าห้องเดียวกัน (JOIN_ROOM)	82ms	ผ่าน (Paired)	
TC-WS-03	E2E Multi-User	ซิงค์การลากวางการ์ด (CARD_MOVED)	64ms	ผ่าน (<100ms)	
TC-WS-04	E2E Multi-User	ซิงค์สถานะหมุนการ์ด (CARD_EXERTED)	58ms	ผ่าน (<100ms)	
TC-WS-05	E2E Multi-User	ซิงค์แต้ม Lore Counter (LORE_UPDATED)	52ms	ผ่าน (<100ms)	
TC-WS-06	Fault Tolerance	จำลองเน็ตหลุดและกู้คืน (60s Rejoin Grace Period)	310ms	ผ่าน (Re-stored)	
TC-WS-07	Lifecycle	ผู้เล่นกดยอมแพ้ (Voluntary Exit Match)	65ms	ผ่าน (Cleared)	
TC-ELAST-01	Stress/AutoScale	ยิงโหลด 2,000 Requests Concurrency 20 ไปยัง ALB	Avg 48ms	ผ่าน (0.00% Err)	

สรุปผลสัมฤทธิ์การทดสอบคุณภาพ:

1. Unit & Store Integration Tests (Vitest 4.x): ผ่านครบ 33 / 33 การทดสอบ (100%)
2. End-to-End Browser Tests (Playwright): ผ่าน 49 / 50 การทดสอบ (98%) บน Chromium
3. Load Stress Test บน ALB จริง (scripts/stress_test.ps1):
 - ยิงโหลด 2,000 HTTP Requests แบบ Concurrency 20 ไปยัง ALB: `lorcana-alb-687861613.us-east-1.elb.amazonaws.com`
 - ผลการทดสอบ: HTTP 200 OK ครบ 2,000 คำขอ (Error Rate 0.00%)
 - CloudWatch Alarm `cpu-target-tracking-60` เข้าสู่สถานะ `In alarm` และ ASG สปอร์น EC2 Instance เครื่องที่ 2 ข้าม Availability Zone ขึ้นมาร่วมแบ่งเบาภาระงานได้จริงตามเงื่อนไข Elasticity

บทที่ 5

การประเมินค่าใช้จ่ายบนคลาวด์และแผนงานขั้นต่อไป (Cost Assessment & Future Roadmap)

5.1 การประเมินค่าใช้จ่ายสถาปัตยกรรมคลาวด์ (\$50 Budget Protection)

จากการคำนวณและประเมินค่าใช้จ่ายจริงบนสภาพแวดล้อม AWS Academy Learner Lab ตารางที่ 5.1 แสดงการเปรียบเทียบค่าใช้จ่ายระหว่างสถาปัตยกรรมทั่วไปกับสถาปัตยกรรมที่ได้รับการปรับแต่งในโครงการ:

ตารางที่ 5.1: การเปรียบเทียบและการประเมินค่าใช้จ่ายบนคลาวด์ตลอดภาคการศึกษา

ทรัพยากรคลาวด์	สเปกและการตั้งค่า	ค่าใช้จ่ายปกติ	ค่าใช้จ่ายโครงการ	กลยุทธ์การควบคุมงบประมาณ
VPC & Network	Multi-AZ VPC + IGW	\$32.40/ด. (NAT)	\$0.00	ตัด NAT Gateway ใช้ Chained Security Groups
Compute (EC2)	t3.micro (Amazon Linux)	\$8.40/ด. (24/7)	~\$0.50	ใช้ Scale-to-Zero (lab_stop.ps1) เปิดเฉพาะตอนเดโม
Load Balancer	Application Load Balancer	\$16.20/ด. (24/7)	~\$0.60	เปิด-ปิดตามช่วงเวลาทำแล็บ
Database	Amazon DynamoDB	Free Tier	\$0.00	ข้อมูล < 1GB อยู่ในเกณฑ์ Free Tier
Storage	S3 & Container Images	\$0.10/ด.	\$0.02	Multi-stage Docker ขนาดเบา < 100MB
รวมประมาณการ	ตลอดภาคเรียน	>\$150.00	<\$3.00 – \$5.00	ประหยัดได้กว่า 90% จากงบ \$50

5.2 แผนงานการพัฒนาสำหรับ Stage 3 (Sprint 4 ถึง Sprint 7)

- Sprint 4 (Async Deck Analyzer via Amazon SQS):** นำ Amazon SQS มาใช้งานวิเคราะห์ความสมดุลdeckแบบ Asynchronous พร้อมพัฒนาอัลกอริทึมแนะนำการ์ดที่เข้าหากันตามประเภทหมึกและเคิร์ฟค่าร้าย
- Sprint 5 (Observability, Monitoring & Security Hardening):** ติดตั้ง AWS X-Ray ติดตาม Request Latency ข้าม Nginx, Node.js, และ DynamoDB พร้อมสร้าง CloudWatch Dashboard รวมสถิติ ALB
- Sprint 6 (Load Testing & High Concurrency Simulation):** ดำเนินการทดสอบโหลดขนาดใหญ่ด้วย Artillery จำลองผู้เล่นพร้อมกัน 100–500 ผู้ใช้ และทดสอบความทนทานของ WebSocket Auto-reconnection
- Sprint 7 (Final Report & Video Presentation):** จัดทำรายงานฉบับสมบูรณ์ วิดีโอสาธิตการใช้งานระบบ และเตรียมการนำเสนอโครงการรอบสุดท้าย

5.3 สรุปผลการดำเนินงานและบทเรียนที่ได้รับ (Sprint Retrospective)

การพัฒนาโครงงาน Disney Lorcana PlayLab Cloud ใน Sprint 1 ถึง 3 บรรลุผลสำเร็จ 100% ตามเป้าหมายของ Stage 2 บทเรียนทางวิศวกรรมที่สำคัญ ได้แก่:

1. **การออกแบบเพื่อควบคุมต้นทุน (Cost-Aware Engineering):** การเข้าใจการทำงานของ VPC และ Security Group ทำให้สามารถตัดต้นทุน NAT Gateway ลงได้โดยไม่สูญเสียความปลอดภัย
2. **การรีดสมรรถนะคอนเทนเนอร์บนทรัพยากรจำกัด:** การแยก Build Stage ออกจาก Runtime ใน Docker ช่วยให้สามารถรันทั้งเว็บเซิร์ฟเวอร์และแอปพลิเคชันบน VM ขนาดเล็ก (**t3.micro** 1GB RAM) ได้อย่างราบรื่นและไม่เกิด OOM Crash
3. **การจัดการสถานะระบบกระจาย (Distributed State Management):** การผสมผสานระหว่าง Optimistic UI, Sticky Sessions บน ALB, และ DynamoDB Persistence ช่วยให้ระบบ Real-time WebSockets มีความเสถียรสูงและฟื้นตัวจากเน็ตหลุดได้อย่างรวดเร็ว

บรรณานุกรม (References)

- [1] Ravensburger & Disney. (2023). *Disney Lorcana Trading Card Game Official Rules and Card Database*. [Online]. Available: <https://www.disneylorcana.com>
- [2] Amazon Web Services. (2026). *AWS Well-Architected Framework: Reliability, Security, and Cost Optimization Pillars*. [Online]. Available: <https://aws.amazon.com/architecture/well-architected/>
- [3] Amazon Web Services. (2026). *Amazon Virtual Private Cloud (VPC) User Guide: Security Groups and Subnet Routing*. [Online]. Available: <https://docs.aws.amazon.com/vpc/>
- [4] Amazon Web Services. (2026). *Elastic Load Balancing: Application Load Balancers Guide*. [Online]. Available: <https://docs.aws.amazon.com/elasticloadbalancing/>
- [5] Amazon Web Services. (2026). *Amazon EC2 Auto Scaling User Guide: Target Tracking Scaling Policies*. [Online]. Available: <https://docs.aws.amazon.com/autoscaling/>
- [6] Amazon Web Services. (2026). *Amazon DynamoDB Developer Guide: Core Components and Best Practices for NoSQL Design*. [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/>
- [7] Docker Inc. (2026). *Multi-stage builds: Best practices for Dockerfile*. [Online]. Available: <https://docs.docker.com/build/building/multi-stage/>
- [8] Nginx Inc. (2026). *NGINX Reverse Proxy and WebSocket Proxying Configuration Guide*. [Online]. Available: <https://nginx.org/en/docs/http/websocket.html>
- [9] React Working Group. (2026). *React 19 Documentation: Concurrent Features and Actions*. [Online]. Available: <https://react.dev>
- [10] IETF (Internet Engineering Task Force). (2011). *RFC 6455: The WebSocket Protocol*. [Online]. Available: <https://tools.ietf.org/html/rfc6455>

ภาคผนวก (Appendix)

ภาคผนวก ก: สคริปต์ควบคุมและปรับใช้ระบบบน AWS (scripts/deploy_ec2_vpc_asg.ps1)

```
1 # scripts/deploy_ec2_vpc_asg.ps1 (Excerpt)
2 param (
3     [string]$Region = "us-east-1",
4     [string]$InstanceType = "t3.micro"
5 )
6
7 Write-Host "=== 1. Creating Lean Multi-AZ VPC (10.0.0.0/16) ===" -ForegroundColor
  Cyan
8 $vpcId = (aws ec2 create-vpc --cidr-block 10.0.0.0/16 --query "Vpc.VpcId" --output
  text)
9
10 Write-Host "=== 2. Creating Subnets in us-east-1a and us-east-1b ===" -
  ForegroundColor Cyan
11 $sub1 = (aws ec2 create-subnet --vpc-id $vpcId --cidr-block 10.0.1.0/24 --
  availability-zone us-east-1a --query "Subnet.SubnetId" --output text)
12 $sub2 = (aws ec2 create-subnet --vpc-id $vpcId --cidr-block 10.0.2.0/24 --
  availability-zone us-east-1b --query "Subnet.SubnetId" --output text)
13
14 Write-Host "=== 3. Setting Up Chained Security Groups ($0.00 NAT Cost) ===" -
  ForegroundColor Cyan
15 $albSg = (aws ec2 create-security-group --group-name lorcana-alb-sg --description "
  ALB SG" --vpc-id $vpcId --query "GroupId" --output text)
16 aws ec2 authorize-security-group-ingress --group-id $albSg --protocol tcp --port 80
  --cidr 0.0.0.0/0
17
18 $ec2Sg = (aws ec2 create-security-group --group-name lorcana-ec2-sg --description "
  EC2 SG" --vpc-id $vpcId --query "GroupId" --output text)
19 aws ec2 authorize-security-group-ingress --group-id $ec2Sg --protocol tcp --port 80
  --source-group $albSg
20 aws ec2 authorize-security-group-ingress --group-id $ec2Sg --protocol tcp --port
  3001 --source-group $albSg
21
22 Write-Host "=== 4. Creating Application Load Balancer & Auto Scaling Group ===" -
  ForegroundColor Cyan
23 # Setup ALB, Target Group, Launch Template with LabRole, and ASG with CPU > 60%
  Policy
```

Listing 5.1: คำสั่งหลักในสคริปต์ปรับใช้ Lean Multi-AZ VPC, ALB และ Auto Scaling Group

ภาคผนวก ข: สคริปต์ 1-Click Scale-to-Zero ควบคุมงบประมาณ (scripts/lab_stop.ps1)

```
1 # scripts/lab_stop.ps1 - 1-Click Scale-to-Zero Guardrail
2 Write-Host ">>> Stopping Lab: Scaling ASG to Zero Instances..." -ForegroundColor
  Yellow
3 aws autoscaling update-auto-scaling-group `
```

```
4 --auto-scaling-group-name "lorcana-asg" `
5 --min-size 0 `
6 --desired-capacity 0
7
8 Write-Host ">>> Lab Stopped Successfully. Compute Cost is now $0.00/hour." -
  ForegroundColor Green
```

Listing 5.2: สคริปต์ยุบเครื่อง EC2 เหลือ 0 เพื่อตัดค่าใช้จ่าย Compute เหลือ \$0.00/ชม.